

Number: P3679R0

Date: 2025-05-16

Audience: [Evolution](#)

Author: [Hana Dusíková](#)

- [Motivation](#)
 - [Sink example](#)
 - [Iterator inside multiple layers of concepts example](#)
- [Design](#)
- [Implementation](#)
- [Language impact](#)
- [Proposed changes to wording](#)
 - [Feature test macro](#)

SFINAEable constexpr exceptions

The language should interpret a constexpr exception inside a constraint evaluation as making the constraint evaluate to `false`. Such a failed overload can now cause other overloads to be selected, making those programs well-defined, and allowing constexpr exceptions to be used to fail constraints. If the failed constraint causes the program to not be valid, the exception's error message is a much better error to report than the instantiation path.

Motivation

Following example shows how currently (C++26) exceptions thrown during evaluation of constraint check results in an error to compile whole program. And what would be result if constexpr exception would result only in SFINAE.

```

template <typename... Ts> constexpr bool throw_reason(std::format_string<Ts...> f,
Ts &&... args) {
    throw exception{std::format(f, std::forward<Ts>(args)...)};
}

template <typename T> concept fits_into_four_bytes = sizeof(T) <= 4 || throw_reas
on("provided type `{}` is larger than 4 bytes!", display_string_of(^T));

int overload(special_type value); // special match
int overload(fits_into_four_bytes auto value);

int main() {
    overload(special_type{}); // totally fine
    overload(42);             // also fine
    overload(new int);
    // BEFORE: error: substitution into constraint
    // expression resulted in a non-constant expression
    // AFTER: can't select any overload from candidates:
    // `int overload(special_type value);`
    //   reason: not matching type
    // `int overload(fits_into_four_bytes auto value);`
    //   reason: provided type `void*` is larger than 4 bytes!
}

```

[Compiler Explorer](#)

Sink example

Similar example as previous, but with additional overload. Notice how previously throwing in constraint check makes the whole overload set unusable. With this paper, it will compile as one would expect.

```

int overload(special_type value); // special match
int overload(fits_into_four_bytes auto value);
int overload(auto &&); // sink with lowest priority

overload(ptr);
// BEFORE: substitution into constraint expression resulted in a non-constant expression
// AFTER: ok, it selected `int overload(auto &&)` "sink" overload

```

Iterator inside multiple layers of concepts example

```

struct hana_super_awesome_container {
    // ...
    hana_iterator begin();
    hana_sentinel end();
};

my_container my_obj{...};

std::ranges::reverse(obj);
// BEFORE: a multiple pages long error messages which
//         will cut the important reason due error depth limit.
// AFTER: simple error: provided iterator `hana_iterator` is not decrementable

```

I'm aware some compilers are getting better in error reporting with concepts. Point is if a library author can provide a custom simple explanation why something is not working it will probably be always better than anything compiler can do automatically.

Design

All outermost constraint are evaluated like they are wrapped in try-catch which returns false for any exception thrown from the body of the constraint.

```

template <template-arguments> concept name = [] {
    try {
        return <EXPR>;
    } catch (...) {
        store-error-message(exception.what());
        return false;
    }
};

```

This makes programs valid in presence of alternative overload candidate. This is useful to introduce sink overloads and also provide error messages in case there is no alternative overload candidate as a reason why specific constraint check failed or overload candidate failed.

Implementation

[Implemented](#) as an extension for constexpr exception in Clang.

Language impact

Allows previously not allowed evaluation of constraints checks (which lead compilation failure).

Proposed changes to wording

13.5.2.3

Atomic constraints

[temp.constr.atomic]

- 1 An *atomic constraint* is formed from an expression *E* and a mapping from the template parameters that appear within *E* to template arguments that are formed via substitution during constraint normalization in the declaration of a constrained entity (and, therefore, can involve the unsubstituted template parameters of the constrained entity), called the *parameter mapping* ([temp.constr.decl]).

[Note 1: Atomic constraints are formed by [constraint normalization](#). *E* is never a [logical AND expression](#) nor a [logical OR expression](#). — end note]

- 2 Two atomic constraints, e_1 and e_2 , are *identical* if they are formed from the same appearance of the same [expression](#) and if, given a hypothetical template *A* whose *template-parameter-list* consists of [template-parameters](#) corresponding and equivalent ([temp.over.link]) to those mapped by the parameter mappings of the expression, a *template-id* naming *A* whose *template-arguments* are the targets of the parameter mapping of e_1 is the same ([temp.type]) as a *template-id* naming *A* whose *template-arguments* are the targets of the parameter mapping of e_2 .

[Note 2: The comparison of parameter mappings of atomic constraints operates in a manner similar to that of declaration matching with alias template substitution ([temp.alias]).

[Example 1:

```
template <unsigned N> constexpr bool Atomic = true;
template <unsigned N> concept C = Atomic<N>;
template <unsigned N> concept Add1 = C<N + 1>;
template <unsigned N> concept AddOne = C<N + 1>;
template <unsigned M> void f()
    requires Add1<2 * M>;
template <unsigned M> int f()
    requires AddOne<2 * M> && true;
```

```
int x = f<0>();           // OK, the atomic constraints from concept C in both fs are Atomic<N>
                          // with mapping similar to  $N \mapsto 2 * M + 1$ 
```

```
template <unsigned N> struct WrapN;
template <unsigned N> using Add1Ty = WrapN<N + 1>;
template <unsigned N> using AddOneTy = WrapN<N + 1>;
template <unsigned M> void g(Add1Ty<2 * M> *);
template <unsigned M> void g(AddOneTy<2 * M> *);
```

```
void h() {
    g<0>(nullptr);    // OK, there is only one g
}
```

— end example]

As specified in [temp.over.link], if the validity or meaning of the program depends on whether two constructs are equivalent, and they are functionally equivalent but not equivalent, the program is ill-formed, no diagnostic required.

[Example 2:

```
template <unsigned N> void f2()
    requires Add1<2 * N>;
template <unsigned N> int f2()
    requires Add1<N * 2> && true;
void h2() {
    f2<0>();                // ill-formed, no diagnostic required:
                           // requires determination of subsumption between atomic constraints
    that are
                           // functionally equivalent but not equivalent
}
```

— end example]

— end note]

- 3 To determine if an atomic constraint is *satisfied*, the parameter mapping and template arguments are first substituted into its expression. If substitution results in an invalid type or expression in the immediate context of the atomic constraint ([temp.deduct-general]), the constraint is not satisfied. Otherwise, the *lvalue-to-rvalue conversion* is performed if necessary, and E shall be a constant expression of type `bool`. If an uncaught exception is thrown during constant evaluation ([expr.const]) of a constraint then the outermost constraint is evaluated as false. The constraint is satisfied if and only if evaluation of E results in `true`. If, at different points in the program, the satisfaction result is different for identical atomic constraints and template arguments, the program is ill-formed, no diagnostic required.

[Example 3:

```
template<typename T> concept C =
    sizeof(T) == 4 && !true;    // requires atomic constraints sizeof(T) == 4 and
    !true

template<typename T> struct S {
    constexpr operator bool() const { return true; }
};

template<typename T> requires (S<T>{})
void f(T);                    // #1
void f(int);                  // #2
```

```

void g() {
    f(0);           // error: expression S<int>{} does not have type b
    ool
}                  // while checking satisfaction of deduced arguments
of #1;

// call is ill-formed even though #2 is a better match

```

— end example]

[Note 3: The compiler can access exception's message via member function what() to print a useful error message to user.

[Example 4:

```

constexpr bool throw_an_exception() {
    throw std::exception("provided type has different size than a pointer");
}

```

```

template <T> concept same_size_as_pointer = sizeof(T) == sizeof(void*) |
| throw_an_exception();

```

```

static_assert(!same_size_as_pointer<char>); // OK, char has different size than a pointer

```

```

void overload(same_size_as_pointer auto v);

```

```

overload('x'); // Error: "provided type has different size than a pointer"

```

— end example]

— end note]

Feature test macro

15.11 Predefined macro names [cpp.predefined]

__cpp_constexpr_exceptions 202411L202???L